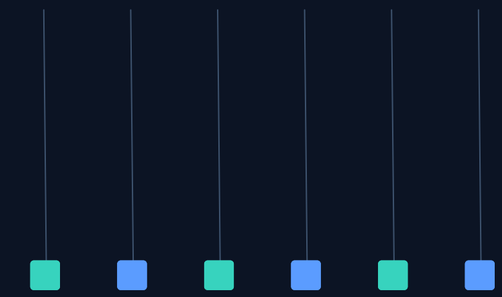
CS203 / DIGITAL DESIGN PROJECT

UART Serial Communication System

Implementation progress review



UART

8-bit serial communication system

8-bit data • TX/RX buffering • FSM control • Simulation-led verification

Group 5
Members:
2025CSB1223 • 2025CSB1204 • 2025CSB1195 • 2025CSB1220 • 2025CSB1221

From proposal to working modules

This review focuses on implementation progress, not just the proposed idea.

Implemented

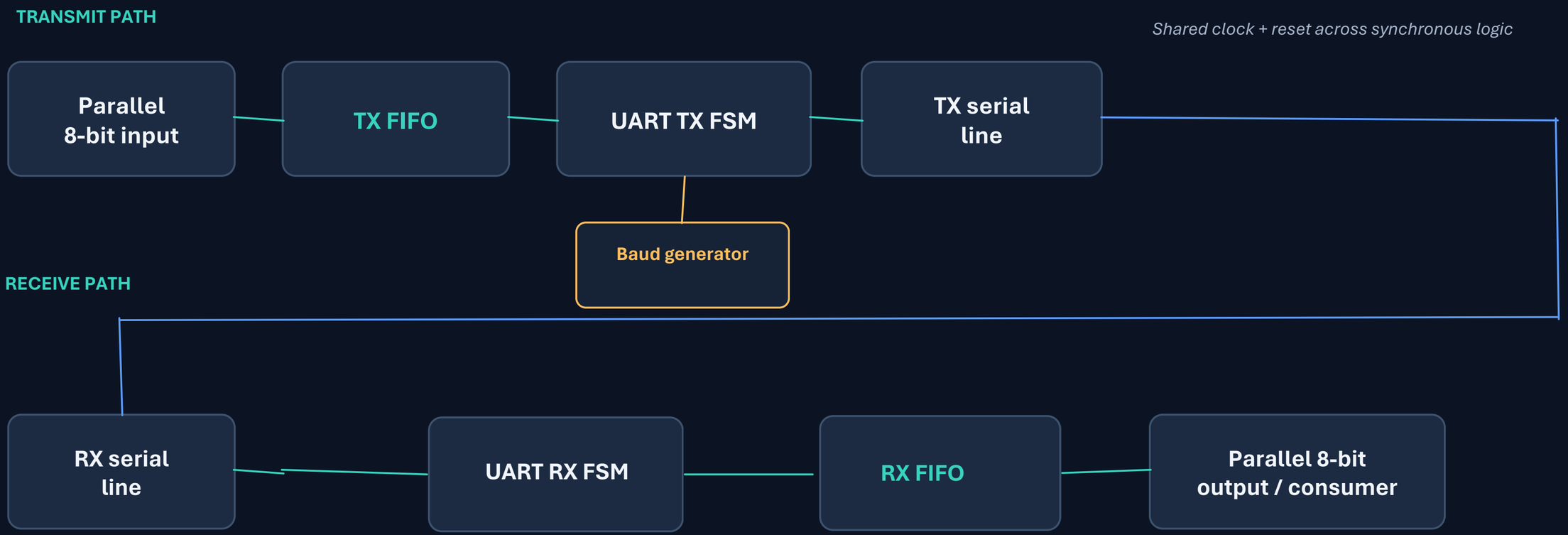
- 16 × 8-bit FIFO
- FSM
- Baud Rate Generator
- Testbench and waveform checks
- UART RX/TX
- Top-level integration

Project target

Transmit and receive 8-bit UART frames reliably, buffer data on both sides, and verify operation using simulation / loopback.

Design note: CLKS_PER_BIT must be calculated from the system clock and baud rate; 16 clock cycles per bit is a test setting, not automatically 16× oversampling.

System architecture



How the design is being built

Modular RTL makes each block easier to test before full-system integration.

01

Specify

Define frame format, clock, baud rate, reset polarity, and FIFO depth.

02

Implement

Write each block as a separate Verilog module with explicit ports.

03

Unit-test

Test normal operation and boundary cases with a focused testbench.

04

Integrate

Connect TX FIFO->UART TX and UART TX->UART RX through loopback.

05

System-test

Run loopback, inspect waveforms, and record pass/fail evidence.

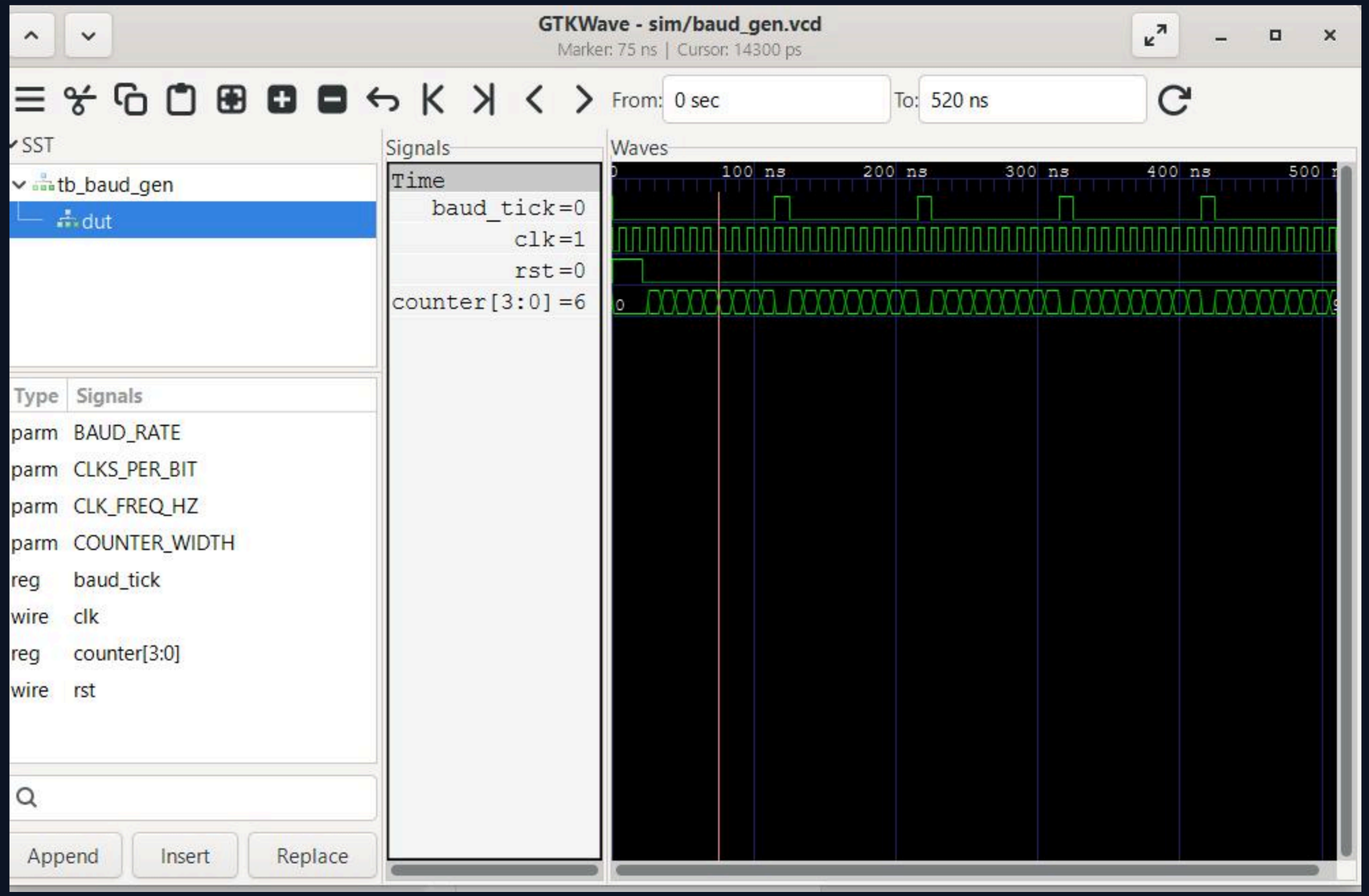
Acceptance principle: each module passes its own tests before its output is trusted by the next block.

RTL modules and key behavior

BLOCK	IMPLEMENTATION	KEY CHECKS	STATUS
FIFO (16 × 8)	Memory array, read/write pointers, occupancy counter, full/empty flags.	No write when full; no read when empty; ordering preserved.	RTL drafted
UART RX	FSM: IDLE → START → DATA → STOP; captures 8 LSB-first bits.	Start validation; bit timing; stop-bit validation; rx_done pulse.	RTL drafted
UART TX	Serializes one byte with start/data/stop framing.	Bit order, bit period, idle-high output.	RTL drafted
Baud generator	Clock divider / tick source for TX and RX timing.	Tick period matches selected clock and baud rate.	RTL drafted
Top integration	Integrates TX FIFOs, UART FSMs, and baud generator to enable serial data transmission and reception.	No lost/duplicated bytes under expected traffic.	RTL drafted

Status is a draft snapshot; edit it to match the latest implementation and verified commits.

Simulation results by component



Baud Rate Generator

Verified periodic baud tick generation for the configured clock frequency and baud rate, ensuring consistent timing for serial bit transmission and reception.

Simulation results by component – Baud Rate Generator

testbench.sv

```
1  `timescale 1ns/1ps
2
3  module tb_baud_gen;
4
5      reg clk;
6      reg rst;
7      wire baud_tick;
8
9      localparam integer EXPECTED_CLKS_PER_BIT = 10;
10
11     integer cycles;
12     integer last_tick_cycle;
13     integer tick_count;
14
15     baud_gen #(
16         .CLK_FREQ_HZ(100),
17         .BAUD_RATE(10)
18     ) dut (
19         .clk(clk),
20         .rst(rst),
21         .baud_tick(baud_tick)
22     );
```

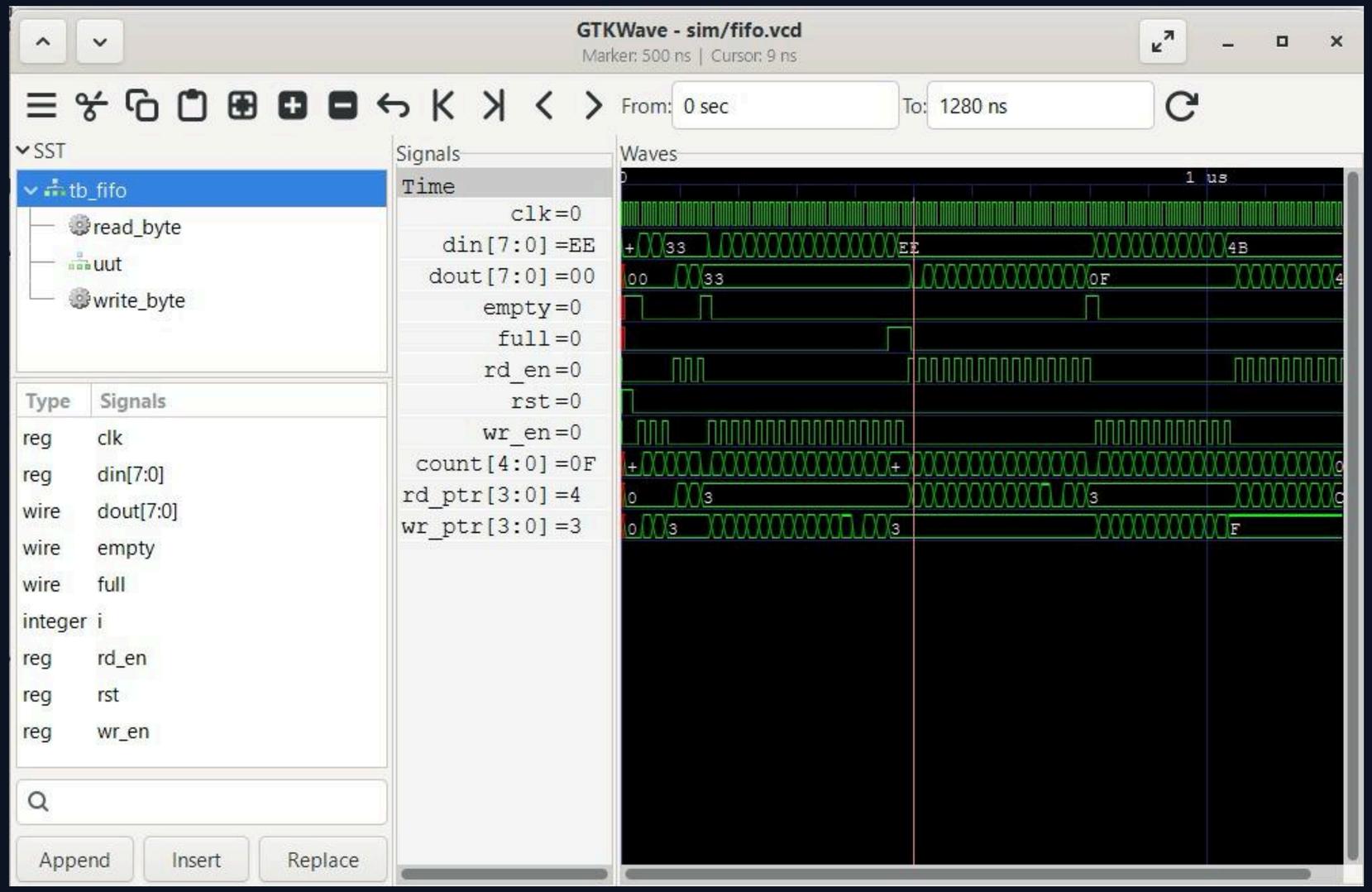
design.sv

```
1  `timescale 1ns/1ps
2  module baud_gen #(
3      parameter integer CLK_FREQ_HZ = 50000000,
4      parameter integer BAUD_RATE = 9600
5  )(
6      input wire clk,
7      input wire rst,
8      output reg baud_tick
9  );
10
11     localparam integer CLKS_PER_BIT = CLK_FREQ_HZ / BAUD_RATE;
12
13     localparam integer COUNTER_WIDTH =
14         (CLKS_PER_BIT <= 1) ? 1 : $clog2(CLKS_PER_BIT);
15
16     reg [COUNTER_WIDTH-1:0] counter;
17
18     always @(posedge clk or posedge rst) begin
19         if (rst) begin
20             counter   <= 0;
21             baud_tick <= 0;
22         end
```

Log | Share

```
[2026-10-11 16:43:22 UTC] iverilog '-Wall' '-g2012' design.sv testbench.sv  && unbuffer vvp a.out
VCD info: dumpfile baud_gen.vcd opened for output.
Tick 1 at cycle 10
Tick 2 at cycle 20
Tick 3 at cycle 30
Tick 4 at cycle 40
Tick 5 at cycle 50
PASS: Baud generator timing is correct
testbench.sv:62: $finish called at 520000 (1ps)
Done
```

Simulation results by component



FIFO — unit tests

Tested write and read operations, data ordering, pointer updates, and correct full/empty flag behaviour, including handling of boundary conditions.

Simulation results by component – FIFO Buffer

testbench.sv

```
1
2  `timescale 1ns/1ps
3
4  module tb_fifo;
5
6      reg clk;
7      reg rst;
8      reg wr_en;
9      reg rd_en;
10     reg [7:0] din;
11
12     wire [7:0] dout;
13     wire full;
14     wire empty;
15
16     integer i;
17
18     // Instantiate the FIFO module
19     fifo uut (
20         .clk(clk),
21         .rst(rst),
22         .wr_en(wr_en),
```

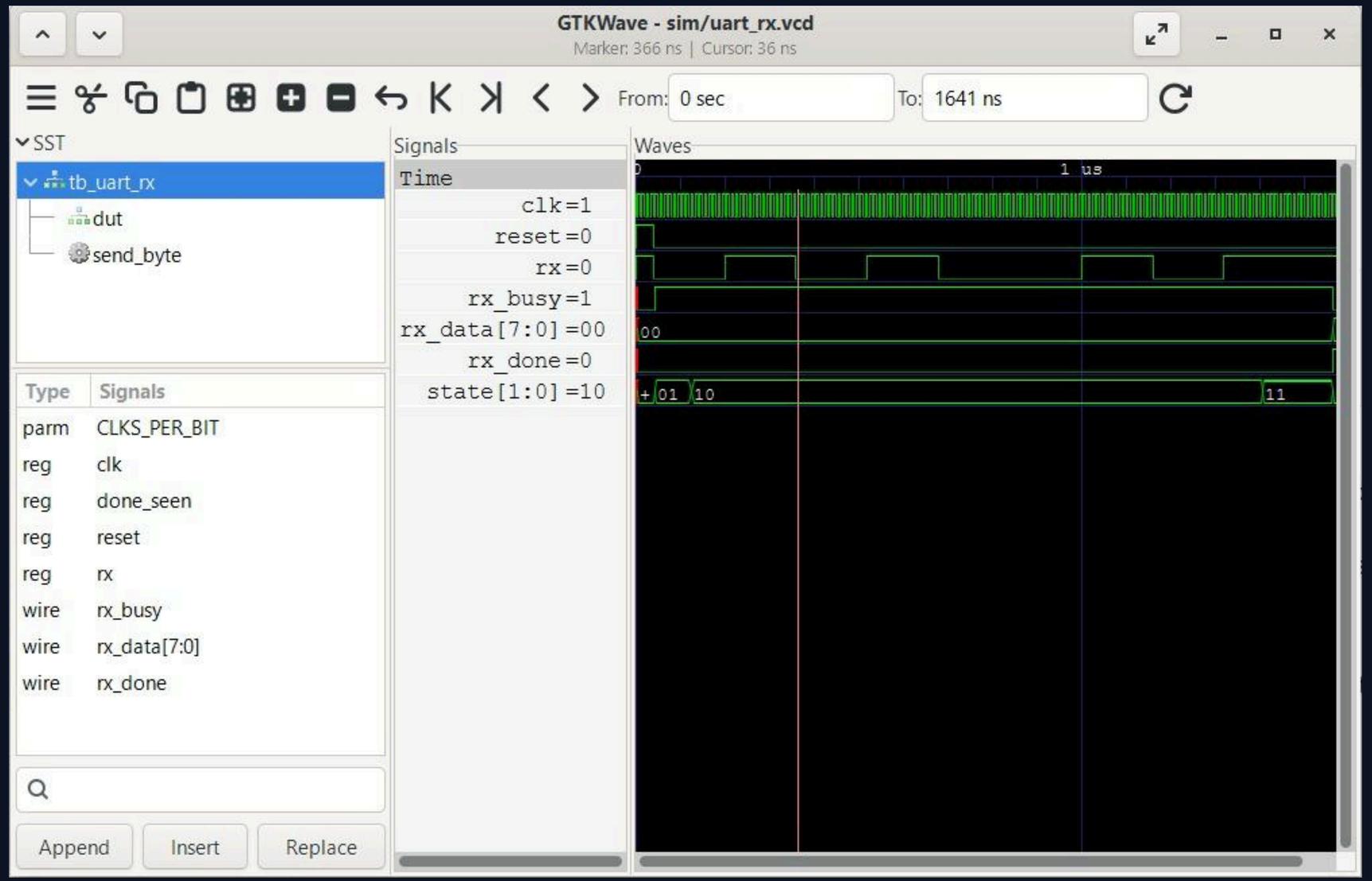
design.sv

```
1
2  `timescale 1ns/1ps
3
4  module fifo (
5      input wire clk,
6      input wire rst,
7
8      input wire wr_en,
9      input wire rd_en,
10
11     input wire [7:0] din,
12     output reg [7:0] dout,
13
14     output wire full,
15     output wire empty
16 );
17     reg [7:0] mem [0:15];
18
19     reg [3:0] wr_ptr;
20     reg [3:0] rd_ptr;
21
22     reg [4:0] count;
```

Log | Share

```
PASS: read 44
PASS: read 45
PASS: read 46
PASS: read 47
PASS: read 48
PASS: read 49
PASS: read 4a
PASS: read 4b
PASS: pointer wraparound
--------------------------------
ALL FIFO TESTS PASSED
--------------------------------
testbench.sv:171: $finish called at 1280000 (1ps)
Done
```

Simulation results by component



UART RX — unit tests

Verified start-bit detection, mid-bit start validation, 8-bit data sampling, stop-bit checking, and correct reconstruction of received bytes.

Simulation results by component – UART RX

testbench.sv

```
1
2  `timescale 1ns/1ps
3
4  module tb_uart_rx;
5
6      reg clk;
7      reg reset;
8      reg rx;
9
10     wire [7:0] rx_data;
11     wire rx_done;
12     wire rx_busy;
13
14     reg done_seen;
15
16     localparam CLKS_PER_BIT = 16;
17
18     // Connect the receiver module to the testbench
19     uart_rx #(
20         .CLKS_PER_BIT(CLKS_PER_BIT)
21     ) dut (
22         .clk(clk),
```

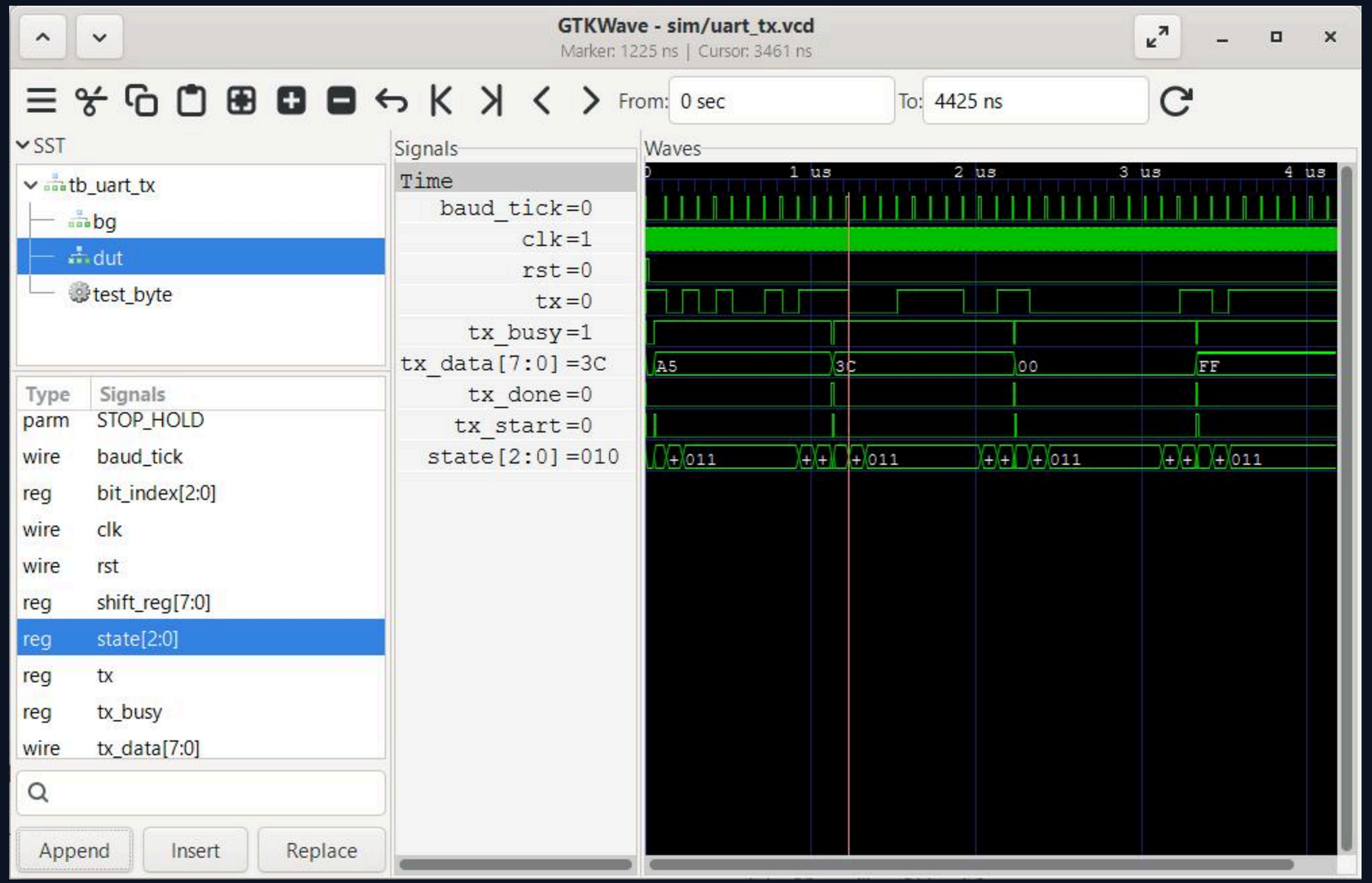
design.sv

```
1  `timescale 1ns/1ps
2  module uart_rx #(
3      parameter CLKS_PER_BIT = 16
4  )(
5      input clk,
6      input reset,
7      input rx,
8
9      output reg [7:0] rx_data,
10     output reg rx_done,
11     output reg rx_busy
12 );
13
14     localparam IDLE  = 2'b00;
15     localparam START = 2'b01;
16     localparam DATA  = 2'b10;
17     localparam STOP  = 2'b11;
18
19     reg [1:0] state;
20     reg [7:0] shift_reg;
21     reg [2:0] bit_index;
22     integer clk_count;
```

Log | Share

```
[2026-10-11 16:41:55 UTC] iverilog '-Wall' '-g2012' design.sv testbench.sv  && unbuffer vvp a.out
VCD info: dumpfile uart_rx.vcd opened for output.
PASS: RX received A5 correctly
testbench.sv:87: $finish called at 1641000 (1ps)
Done
```

Simulation results by component



UART TX

Verified transmission of start, 8 data, and stop bits in the correct sequence, with appropriate bit timing and serial output transitions.

Simulation results by component – UART TX

testbench.sv

```
// Code your testbench here
// or browse Examples
`timescale 1ns/1ps

module tb_uart_tx;

    reg clk;
    reg rst;
    reg tx_start;
    reg [7:0] tx_data;

    wire baud_tick;
    wire tx;
    wire tx_busy;
    wire tx_done;

    integer i;
    reg [7:0] received;

    baud_gen #(
        .CLK_FREQ_HZ(100),
        .BAUD_RATE(10)
    ) bg (
        .clk(clk)
```

design.sv

```
// Code your design here

`timescale 1ns/1ps
module uart_tx(
    input wire clk,
    input wire rst,
    input wire tx_start,
    input wire [7:0] tx_data,
    input wire baud_tick,
    output reg tx,
    output reg tx_busy,
    output reg tx_done
);

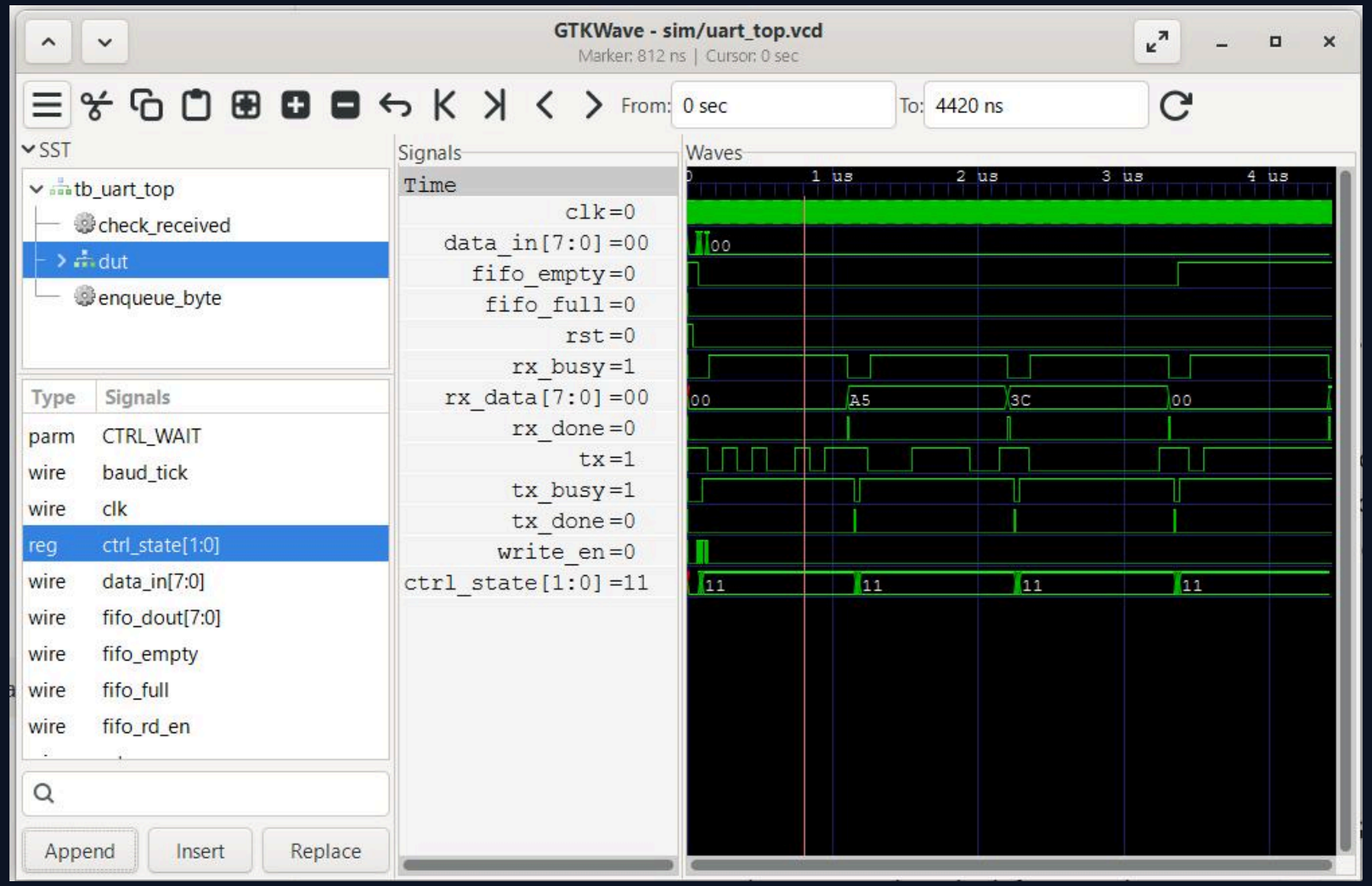
    localparam IDLE      = 3'd0;
    localparam START_WAIT = 3'd1;
    localparam START_BIT = 3'd2;
    localparam DATA      = 3'd3;
    localparam STOP_BIT  = 3'd4;
    localparam STOP_HOLD = 3'd5;

    reg [2:0] state;
    reg [7:0] shift_reg;
    reg [2:0] bit_index;
```

Log | Share

```
[2026-10-11 16:37:13 UTC] iverilog '-Wall' '-g2012' design.sv testbench.sv  && unbuffer vvp a.out
VCD info: dumpfile uart_tx.vcd opened for output.
PASS: TX=a5 RX=a5
PASS: TX=3c RX=3c
PASS: TX=00 RX=00
PASS: TX=ff RX=ff
ALL UART TX TESTS PASSED
testbench.sv:109: $finish called at 4425000 (1ps)
Done
```

Simulation results by component



UART Top Level Integration

Verified communication between all modules through loopback testing, including FIFO-to-transmitter flow, serial reception, receiver-to-FIFO transfer, and correct recovery of transmitted data.

Simulation results by component – Top Level Integration

testbench.sv

```
1  `timescale 1ns/1ps
2
3  module tb_uart_top;
4
5      localparam CLK_FREQ_HZ = 100;
6      localparam BAUD_RATE = 10;
7
8      reg clk;
9      reg rst;
10     reg [7:0] data_in;
11     reg write_en;
12
13     wire fifo_full;
14     wire fifo_empty;
15     wire tx;
16     wire tx_busy;
17     wire tx_done;
18     wire [7:0] rx_data;
19     wire rx_done;
20     wire rx_busy;
21
22     integer cycles;
```

design.sv

```
1  `timescale 1ns/1ps
2  module uart_top #(
3      parameter integer CLK_FREQ_HZ = 50000000,
4      parameter integer BAUD_RATE = 9600
5  )(
6      input wire clk,
7      input wire rst,
8
9      input wire [7:0] data_in,
10     input wire write_en,
11     output wire fifo_full,
12     output wire fifo_empty,
13
14     output wire tx,
15     output wire tx_busy,
16     output wire tx_done,
17
18     input wire rx,
19     output wire [7:0] rx_data,
20     output wire rx_done,
21     output wire rx_busy
22 );
```

Log | Share

```
[2026-10-11 16:47:13 UTC] iverilog '-Wall' '-g2012' design.sv testbench.sv  && unbuffer vvp a.out
VCD info: dumpfile uart_top.vcd opened for output.
Sending bytes through FIFO and UART loopback...
PASS: Expected=a5, Received=a5
PASS: Expected=3c, Received=3c
PASS: Expected=00, Received=00
PASS: Expected=ff, Received=ff
---------------------------------
ALL UART TOP TESTS PASSED
---------------------------------
testbench.sv:127: $finish called at 4420000 (1ps)
Done
```

Issues encountered and how we are addressing them

Timing / sampling	Align start-bit validation and data sampling to the bit centre; confirm CLKS_PER_BIT from clock and baud.
FIFO full behavior	A full FIFO rejects writes; and empty FIFO rejects read.
Integration timing	Registered FIFO output and one-cycle rx_done pulses require clear handshakes and waveform inspection.
Module Integration Issues:	Differences in signal timing and control between modules could affect data transfer. Tested modules individually before integrating them and verified end-to-end communication through loopback testing.
Simulation setup:	Tool configuration issues were resolved by setting up Icarus Verilog, VVP, and GTKWave for simulation and debugging.
Transmission errors:	Parity bit checking is planned to detect errors, with Hamming code considered for error correction.

Work completed by each member

Gunjan Chaudhary – 2025CSB1195

FIFO Buffer, Top level integartion and Waveform analysis

Implemented buffer storage, read/write pointers, occupancy counter, and full/empty flag logic.

Kanchi Aggarwal – 2025CSB1204

UART RX , Top level integartion and Waveform analysis

Designed the RX FSM for start-bit detection, 8-bit data sampling, stop-bit validation, and byte reconstruction.

Mishra Rishikesh – 2025CSB1223

UART TX, Top level integartion and Waveform analysis

Designed the TX FSM for start-bit generation, serial transmission of 8-bit data, and stop-bit generation.

Meet Kamlesh Trivedi – 2025CSB1220

Testbench, Top level integration

Connected all modules, and developed testbenches to verify end-to-end functionality.

Menkudle Shreyash Sudhir – 2025CSB1221

Baud Generator, Top level integartion and Waveform analysis

Implemented clock division and baud-tick generation for synchronized TX/RX bit timing.

Version control and collaboration

GITHUB REPOSITORY

https://github.com/mishrarishikesh333/UART_Project.git

- Branch: [main] • RTL: /rtl • Simulation: /sim • Testbench:/tb • Project Report:/docs